

Computer-Assisted Formalization of Mathematics

Week 03 - Dependent types and universes

First Half

- ▶ Π -types and Σ -types
- ▶ Universes and universe polymorphism
- ▶ The three axioms of Lean

Second Half

- ▶ FunExt
- ▶ $P \vee \neg P$

Jaume de Dios Pont - jdedios@nyu.edu

<https://nyulean26.github.io/>

Dependent Type Theory

Universes

The 3 axioms of Lean

Why Dependent Types? (Motivation I) – Data indexed by values

Goal 1: define a function that, for each n , returns a length n vector full of ones.

Types: $(n : \mathbb{N}), \quad \text{Vec } \mathbb{N} \ n$ – integer vectors of length n .

- Type of the result $\text{Vec}(\mathbb{N}, n)$ *depends* on the input value n .

Why Dependent Types? (Motivation I) – Data indexed by values

Goal 1: define a function that, for each n , returns a length n vector full of ones.

Types: $(n : \mathbb{N})$, $\text{Vec } \mathbb{N} \ n$ – integer vectors of length n .

- Type of the result $\text{Vec}(\mathbb{N}, n)$ *depends* on the input value n .

```
def repeated_ones : (n : ℕ) → Vector ℕ n :=  
  fun n ↦ Vector.replicate n 1
```

Why Dependent Types? (Motivation I) – Data indexed by values

Goal 1: define a function that, for each n , returns a length n vector full of ones.

Types: $(n : \mathbb{N})$, $\text{Vec } \mathbb{N} \ n$ – integer vectors of length n .

- Type of the result $\text{Vec}(\mathbb{N}, n)$ *depends* on the input value n .

```
def repeated_ones : (n : ℕ) → Vector ℕ n :=  
  | fun n ↦ Vector.replicate n 1
```

Goal 2: Define a function that for each $a : A$, shows that the property P_a holds.

- Type of the proof depends on a (and b).

Why Dependent Types? (Motivation I) – Data indexed by values

Goal 1: define a function that, for each n , returns a length n vector full of ones.

Types: $(n : \mathbb{N})$, $\text{Vec } \mathbb{N} \ n$ – integer vectors of length n .

- Type of the result $\text{Vec}(\mathbb{N}, n)$ *depends* on the input value n .

```
def repeated_ones : (n : ℕ) → Vector ℕ n :=  
  fun n ↦ Vector.replicate n 1
```

Goal 2: Define a function that for each $a : A$, shows that the property P_a holds.

- Type of the proof depends on a (and b).

```
-- `nat_zero_add` is a "map" from (a : ℕ) to a term of "0+a=a".  
def nat_zero_add : (a : ℕ) → (0+a=a) := zero_add  
  
-- ∀ are coerced to this "maps" (Π-Types)  
example : ∀ (a b : ℕ), a+b=b+a := Nat.add_comm
```

Π -Types (Dependent Function Types)

Idea.

- ▶ Generalizes $A \rightarrow B$: the codomain may depend on the input.
- ▶ Formal: $\Pi(x : A), B(x)$, where $B : A \rightarrow \text{Type}$.
- ▶ Read: “for each $x : A$, give an element of $B(x)$ ”.

Math intuition.

A family of types $B(x)$ indexed by $x \in A$;
a dependent function picks $b_x \in B(x)$ for each x .

Examples.

- ▶ $n \mapsto \text{Vector}(\mathbb{N}, n)$ (data type indexed by n).
- ▶ $\forall n : \mathbb{N}, \underbrace{n + 0 = n}_{\in \text{Prop}}$ (theorem type indexed by n).

Σ -Types (Dependent Pair Types)

Idea.

- ▶ Generalizes $A \times B$: the second component's type may depend on the first.
- ▶ Formal: $\Sigma(x : A), B(x)$, where $B : A \rightarrow \text{Type}$.
- ▶ Read: “a pair $\langle x, b \rangle$ with $x : A$ and $b : B(x)$ ”.

Math intuition.

Pairs where the second entry's type is a fiber over the first:

$$\langle x, b \rangle \in \bigcup_{x \in A} \{x\} \times B(x).$$

Examples.

- ▶ “There exists $x : \mathbb{R}$ with $x > 0$ ” as a pair $\langle x, \text{proof}(x > 0) \rangle$.
- ▶ Nonnegative reals as $\Sigma(x : \mathbb{R}), x \geq 0$.

Subtypes

Definition.

$$\{x : A \mid P(x)\} \cong \langle x : A, P(x) \rangle$$

Reading.

- ▶ “Elements of A equipped with a witness that they satisfy P ”.
- ▶ Carries both the value and the evidence of the property.

Examples.

- ▶ Even naturals: $\{x : \mathbb{N} \mid \exists k, x = 2k\}$.
- ▶ Domain-restricted objects: $\{x : \mathbb{R} \mid x \geq 0\}$ for totalized $\sqrt{}$.
- ▶ Bounded sets, Continuous functions, Lipschitz maps with constant L , etc.

Structures as Σ -types

In practice we often use structures to present Σ -types nicely.

```
structure NatPoint where  
  x : ℕ  
  y : ℕ
```

Structures as Σ -types

In practice we often use structures to present Σ -types nicely.

```
structure NatPoint where  
  x : ℕ  
  y : ℕ
```

```
def p := NatPoint.mk 2 3
```

```
def p' : NatPoint := ⟨3,4⟩
```

```
def q := NatPoint where  
  x := 2  
  y := 3
```

Structures as Σ -types

In practice we often use structures to present Σ -types nicely.

```
structure NatPoint where
```

```
  x : ℕ
```

```
  y : ℕ
```

```
def p := NatPoint.mk 2 3
```

```
def p' : NatPoint := ⟨3,4⟩
```

```
def q := NatPoint where
```

```
  x := 2
```

```
  y := 3
```

```
structure Semigroup' where
```

```
  carrier : Type*
```

```
  mul : carrier → carrier → carrier
```

```
  mul_assoc : ∀ a b c, mul (mul a b) c = mul a (mul b c)
```

Dependent Type Theory

Universes

The 3 axioms of Lean

Did we do something illegal?

We have defined:

1. $\Sigma(x : A), B(x)$, where $B : A \rightarrow \text{Type}$
2. $\Pi(x : A), B(x)$, where $B : A \rightarrow \text{Type}$

Who is “Type”? The type of all types?

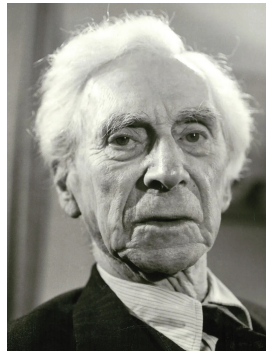
Did we do something illegal?

We have defined:

1. $\Sigma(x : A), B(x)$, where $B : A \rightarrow \text{Type}$
2. $\Pi(x : A), B(x)$, where $B : A \rightarrow \text{Type}$

Who is “Type”? The type of all types?

Russell looks at us with concern



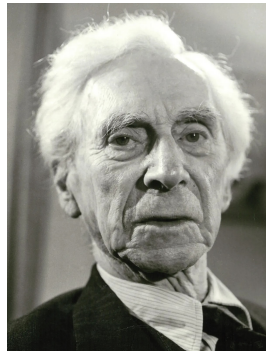
Did we do something illegal?

We have defined:

1. $\Sigma(x : A), B(x)$, where $B : A \rightarrow \text{Type}$
2. $\Pi(x : A), B(x)$, where $B : A \rightarrow \text{Type}$

Who is “Type”? The type of all types?

Russell looks at us with concern



Solution Type is not a “Type”, but a “1-Type”

Girard's paradox

The dangerous combination. Suppose there were one maximal universe U such that

$$U : U, \quad A : U, B : A \rightarrow U \implies \Pi(x : A), B(x) : U.$$

Girard's paradox shows that these assumptions let us construct an inhabitant of an empty type. The resulting logic proves False.

Burali–Forti analogy. If all ordinals formed an ordinal Ω , then Ω would be larger than every ordinal. But Ω would itself be an ordinal, forcing $\Omega < \Omega$.

Lean's solution. There is no maximal universe:

$$\text{Type}_u : \text{Type}_{u+1}, \quad A : \text{Type}_u, B : A \rightarrow \text{Type}_v \implies \Pi(x : A), B(x) : \text{Type}_{\max(u,v)}.$$

Universes in Lean

► Universe Hierarchy

$\text{Prop} : \text{Type}_0, \quad \text{Type}_0 : \text{Type}_1, \quad \text{Type}_1 : \text{Type}_2, \dots$

- Prop is the universe of propositions.
- Each Type_u lives in the next universe Type_{u+1} .

Universes in Lean

► Universe Hierarchy

$$\text{Prop} : \text{Type}_0, \quad \text{Type}_0 : \text{Type}_1, \quad \text{Type}_1 : \text{Type}_2, \dots$$

- Prop is the universe of propositions.
- Each Type_u lives in the next universe Type_{u+1} .
- **Why Universes?**
 - Prevents logical paradoxes (like Girard's paradox).
 - Ensures types cannot refer to themselves in inconsistent ways.

Universes in Lean

► Universe Hierarchy

$$\text{Prop} : \text{Type}_0, \quad \text{Type}_0 : \text{Type}_1, \quad \text{Type}_1 : \text{Type}_2, \dots$$

- Prop is the universe of propositions.
- Each Type_u lives in the next universe Type_{u+1} .
- **Why Universes?**
 - Prevents logical paradoxes (like Girard's paradox).
 - Ensures types cannot refer to themselves in inconsistent ways.
- **Universe Polymorphism**
 - Definitions can be polymorphic over universes using $\{u\}$.

Universe Polymorphism

Theorems should work in $\text{Type}_1, \text{Type}_2, \dots$. How do we prove theorems for all levels?

- ▶ Many library defs carry universe levels:

$$\text{Prod.}\{u, v\}(\alpha : \text{Type}_u)(\beta : \text{Type}_v) : \text{Type}_{\max(u, v)}.$$

- ▶ Typical patterns:
 - ▶ Polymorphic identity: work for any Type_u .
 - ▶ Structures over mixed levels land in $\text{Type}_{\max}$.
 - ▶ Prop vs Type: propositions live in Prop; data lives in some Type_u .

Proof irrelevance and Prop

Question If X is in Type_3 , where should $\forall x : X, P(x)$ live?

Proof irrelevance and Prop

Question If X is in Type_3 , where should $\forall x : X, P(x)$ live?

(a) It's a Prop, so in Prop.

(b) It's made of objects in Type_3 , so in Type_3 .

Proof irrelevance and Prop

Question If X is in Type_3 , where should $\forall x : X, P(x)$ live?

(a) It's a Prop, so in Prop.

(b) It's made of objects in Type_3 , so in Type_3 .

Lean makes one controlled exception to predicativity:

$$A : \text{Sort } u, \quad P : A \rightarrow \text{Prop} \quad \Longrightarrow \quad (\forall x : A, P(x)) : \text{Prop} = \text{Sort } 0.$$

A proposition may quantify over objects from any universe and still return to Sort 0.

- **Proof irrelevance:** any two proofs $p, q : P$ are definitionally equal.
- Proofs carry no runtime data, and elimination from Prop into arbitrary data is restricted.

Proof irrelevance lets us talk about $\exists x, P(x)$ in Prop even if x is not in Prop.

Dependent Type Theory

Universes

The 3 axioms of Lean

The three axioms

Lean's trusted logic has three standard mathematical axioms:

$$\text{propext} : (P \leftrightarrow Q) \rightarrow P = Q,$$
$$\text{Quot.sound} : r\ a\ b \rightarrow \text{Quot.mk}\ r\ a = \text{Quot.mk}\ r\ b,$$
$$\text{Classical.choice} : \text{Nonempty}\ \alpha \rightarrow \alpha.$$

- ▶ `propext` identifies logically equivalent propositions.
- ▶ `Quot.sound` identifies related representatives inside a quotient.
- ▶ `Classical.choice` extracts data from a proof of mere nonemptiness.

We can check the axioms with `#print axioms myTheorem`.

Quot.sound

Start with a type α and any relation $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$. Define the equivalence

$$[a]_r := \text{Quot.mk } r \ a.$$

The soundness axiom is the step that really forms the quotient:

$$h : r \ a \ b \quad \Longrightarrow \quad \text{Quot.sound}(h) : [a]_r = [b]_r.$$

- ▶ Without `Quot.sound`, one could interpret `Quot r` as α and $[a]_r$ as a .
- ▶ Comes with a lift principle $r \ a \ b \rightarrow f(a) = f(b)$; then `Quot.lift(f)` sends $[a]_r$ to $f(a)$.
- ▶ Comes with the rfl equality $\text{Quot.lift}(f) = f \circ \text{Quot.mk}$.

The choice function

The axiom supplies one polymorphic function:

$$\text{Classical.choice} : \prod_{\alpha:\text{Sort } u} (\text{Nonempty}(\alpha) \rightarrow \alpha).$$

Thus a family of nonempty types gives a single function making all the choices:

$$h : \prod_{x:A} \text{Nonempty}(B(x)) \implies (x \mapsto \text{choice}_{B(x)}(h(x))) : \prod_{x:A} B(x).$$

- ▶ Every call applies this same function.
- ▶ If $e : A = B$, transport along e identifies the selected elements:

$$\text{transport}_e(\text{choice}_A(h_A)) = \text{choice}_B(\text{transport}_e(h_A)).$$

- ▶ Proof irrelevance makes the result independent of which proof of $\text{Nonempty}(A)$ we supply.